Introduction to Architecture Exam

ExamTag: 2022-Jun

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Be sure to circle the answers on the page titled "answers page" or your answers will not be graded. If you choose not to answer a question do not circle a letter. Feel free to detach the answers page from the remainder of the exam and only turn it in.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

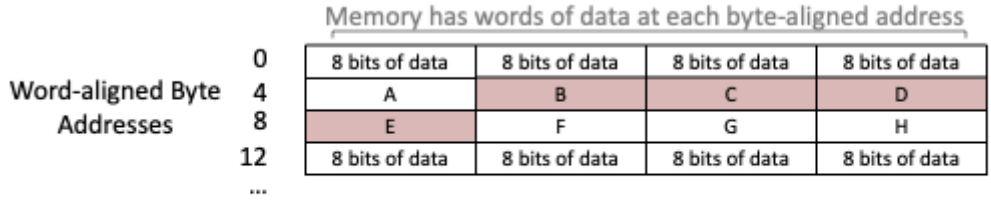
1. How many registers can you use to store data in a MIPS program?
   `2013-Dec-ISA 1-5`
   a. 32
   b. 31
   c. 24
   d. Depends on which registers the function calls use

   We can use any register except R0 to store data. If we use them in the wrong way then we may have compatibility issues, though.

2. What does the following code put in R3?
```
lw R1, 12(R0)
lw R2, 16(R0)
sll R1, R1, 16
slr R2, R2, 16
or R3, R1, R2
```
   `2013-Apr-ISA 1-4`
   a. Zero
   b. The ORed sum of the data at addresses 12 and 16
   c. The ORed sum of the data at addresses 12 and 16 multiplied by 216
   d. The word starting at address 14

   This code loads the data at word-aligned addresses 12 and 16 into R1 and R2, then shifts R1 and R2 such that R1 has the data at bytes 14,15,00000... and R2 has 0000...,16,17. The two are then ORed to produce the un-aligned word from address 14, 15, 16, 17.

3. MIPS does not support unaligned accesses. Below is a list of instructions in order with a brief explaination. Identify the *wrong* instruction or explaination when trying to load bytes B,C,D,E into register R3.



(The unaligned word at address 5 contains bytes B C D E.)

   `2022-Apr-ISA 1-1`
   a. `lw R2, 4(R0)`: load word at address 4
   b. `sll R2, R2, 8`: (Shift by 1 byte)
   c. `lw R4, 8(R0)`: load word at address 8
   d. `or R3, R4, R2`: Combine R4 and R2 using bitwise-OR

   Should be lbu R4, 8(R0). We want to load 'E' into R4, not E,F,G,H.

4. Which register is most likely to be modified?
   `2018-Oct-ISA 1-1`
   a. R0
   b. A0
   c. RA
   d. PC

   The program counter (PC) is modified every time an instruction is executed, so it is most likely to be modified.

5. Why is it less efficient to compute directly on data in memory than data in registers?
   `2015-Aug-ISA 1-1`
   a. Main memory is slower than registers
   b. Main memory requires more address bits to access than registers
   c. A and B
   d. Computing on data in memory is as efficient as data in registers

   The register file is much smaller than main memory so it is much faster to access (1 cycle vs 100 cycles) and we only need a few bits (5 bits for MIPS vs. 32 bits for memory) to address it.

6. In MIPS, what special meaning does the "exit:" label has compared to other labels?
   `2022-Jun-ISA 1-1`
   a. It must be used in branch instructions to jump to the end of a loop
   b. It must be used in branch instructions to jump to the end of the program
   c. a and b
   d. It has no special meaning

   Labels only have the semantic you give them. You are free to choose whatever name you want..

2. ISA 2

1. This is the last code in a procedure. What instructions should be placed in _____?
   lw $t0, 0($sp)
   lw $t1, 4($sp)
   _____
   _____
   `2015-Aug-ISA 2-6`
   a. addi $sp, $sp, 8; jr $ra
   b. addi $sp, $sp, -8; jal proceedure
   c. addi $sp, $sp, 2; jr $ra
   d. addi $sp, $sp, -2; jal proceedure

   We are loading two words from the stack back into $t0 and $t1, so we need to move the stack pointer back UP by 2 words, or 8 bytes, and then do a return jump.

2. How does the processor know the instruction type?
`2017-Jan-ISA 2-4`
   a. If the instruction has 0, 2, or 3 register fields
   b. If there is an immediate in the instruction
   c. The binary code contains instruction type bits for each instruction
   d. None of the above

The processor has to look at the opcode field and decode it. There is no way to know if the bits in the instruction are immediates or register fields without decoding the opcode to know the type.

3. What happens with the immediate for branch instructions, and why?
`2015-Apr-ISA 2-3`
   a. It is shifted right by 8 bits because the lower byte is not necessary
   b. It is shifted right because instruction addresses are always multiples of 4
   c. It is shifted left because instructions are word-aligned
   d. It is not modified

The offsets in branches are in units of 'instructions'. As instructions are 4B each (meaning they are word-aligned), the offset is shifted to the left by 2 bits (effectively multiplying by 4) thus translating from number of instructions to bytes.

4. Select the correct branch instruction that will complete the for loop: `for (i = 0; i != 10; i++)`.

```
add  $t0, $zero, $zero     # i = 0
addi $t1, $zero, 10        # j = 10
addi $t0, $t0, 1           # i++
slt  $t2, $t0, $t1         # t2=1 if (t0<t1) else 0
bne  $t2, $zero, _____     #  jump to ?
```

`2022-Apr-ISA 2-1`
   a. bne $t2, $zero, -8
   b. bne $t2, $zero, -2
   c. bne $t2, $zero, -12
   d. bne $t2, $zero, -3

Branch needs to branch back to the 3rd instruction (which is 2 instructions above). However, as we also need to account for th PC+4 that will already have been applied to the PC, we need to move it 3 instructions backwards, thus the -3

5. A dual-issue pipeline has one ALU that can take inputs from two registers or one register and the sign-extender, and sends its result back to the register file. The second ALU can take inputs from one register and the sign-extender, but only sends its result to the address input of the memory. A designer realizes that this is not ideal and changes it so either ALU can send their results back to the register file. Which pair of instructions can now be executed at the same time that could not be before? (Note, the list below does not imply any ordering. E.g., A and B does not mean that A has to execute on ALU 1 and B on ALU 2.)
`2018-Oct-ISA 2-1`
    a. add and lw
    b. addi and lw
    c. add and addi
    d. lw and lw

    [ALU1=add, ALU2=lw] could be done before. [ALU1=addi, ALU2=lw] could be done before. [ALU1=lw, ALU2=lw] cannot be done on either since both would need to write to the memory at the same time. With the new design, [ALU1=add, ALU2=addi] is now possible since addi writes back to the register file.

6. A function uses registers $s0, $s1 and $t3. How much will it adjust the stack when it is called?
`2013-Aug-ISA 2-6`
    a. -1
    b. -2
    c. -4
    d. -8

    The callee needs to save $s1 and $s0 on the stack, which is two words, or 8 bytes. So the stack needs to move down by -8.

3. Computer Arithmetic

1. Base-2 arithmetic is what is used in our MIPS processor: numbers are stored in binary (1s and 0s) and each binary digit represents twice the value before it. Another approach is to use Base-10 number representations, where we use 4 binary digits together to represent the numbers 0-9. (Representing 0-99 would require two sets of 4 bits, one for the one's digit and one for the ten's digit, and representing 0.0 to 9.9 would require the same number of bits.) We also need to build Base-10 adders that can do the right thing with this representation. Why would one want to use base-10 arithmetic in computers instead of base-2 arithmetic? (Hint: IBM actually makes processors that have hardware for Base-10. Why would they do that?)
`2016-Aug-Computer Arithmetic-2`
    a. It is easier for humans to reason about because we think in Base-10
    b. It is easier for computers to work with because carries are simpler in adders
    c. It is easier to avoid rounding errors with values like money that are already in Base-10
    d. None of the above

    Base-2 arithmetic has rounding rules that look different from base-10 rounding rules. Typically, money and other human inputted data is represented in base-10 which would require extensive checking and overhead if one were to try and emulate the results with base-2 arithmetic.

2. What would the following 2's complement math compute: (A+1) + (!B + !C)
   2014-Dec-Computer Arithmetic-2
   a. A-B-C-1
   b. A-B+C-1
   c. A+B+C-1
   d. A-B+C

   To subtract in 2's complement you need to do A + !B +1. So (A+1)+!B = A-B. Then A-B + !C = (A-B + !C +1) -1 = A-B-C-1

3. How are binary numbers distributed along the number line for the fixed-point number format?
   2015-Apr-Computer Arithmetic-5
   a. Linearly
   b. Exponentially
   c. Depends on the binary point location
   d. Depends on the exponent

   Fixed-point counts just like regular binary, but everything is divided by the binary point shift. E.g., 2.2 counts 0,1,2,3, etc, but everything is divided by 4 (shifted right by two decimal places) so the interpretation is 0.0, 0.25, 0.5, 0.75, etc. Only floating point has exponentially distributed numbers.

4. For a 6-bit normalized floating-point format: (-1)^S*(.FFF)*(2^EE) where FFFF is unsigned and EE is two's complement, what is the smallest positive value you can represent?
   2016-Oct-Computer Arithmetic-2
   a. 0.5
   b. 0.25
   c. 0.125
   d. 0.28125

   0 000 01 = 1*1.0*2^(-2) = 0.25

5. Will the results be correct if you use an ALU designed for 32-bit two's complement addition for 32-bit unsigned integers?
   2015-Oct-Computer Arithmetic-1
   a. Yes, however the overflow flag is calculated differently, and may be incorrect
   b. No, if MSB is 1 for any of the inputs, then the answer will be incorrect
   c. Yes, the same algorithm is used, regardless whether the input happen to be signed or not
   d. No, all signed numbers are subjected to the operation NOT(input) + 1 before an addition, which would produce garbage in the unsigned case

   Two's compliment uses the same logic as unsigned except for overflow.

6. A company develops two new types of transistors: LF transistors are very large, but very fast, and SS transistors are very small but slow. How would you design a multiplier for each type of transistor to match its characteristics?
   `2014-Sep-Computer Arithmetic-2`
   a. LF: serial, SS: serial
   b. LF: serial, SS: parallel
   c. LF: parallel, SS: parallel
   d. LF: parallel, SS: serial

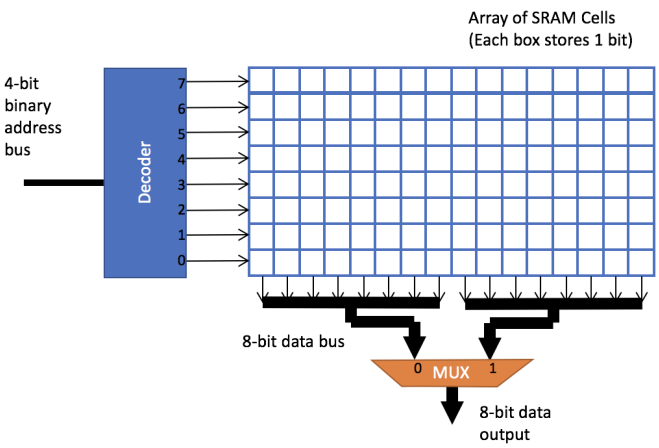
   For LF you want a small circuit, but it is okay if it takes many cycles since they are fast transistors: serial. For SS you can have a large circuit (since they are small) but it shouldn't take many cycles since they are slow: parallel.

4. Logic

1. Why is DRAM so much cheaper than SRAM?
   `2014-Apr-Logic-2`
   a. DRAM doesn't require electricity to retain its values
   b. DRAM memory cells are much smaller than SRAM cells
   c. DRAM doesn't require transistors to work
   d. All of the above

   DRAM is cheaper because it is smaller.

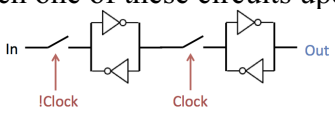
2. What would happen if you replaced the decoder in the SRAM design with an encoder?

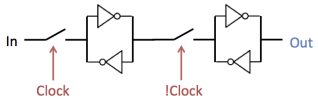


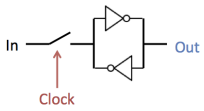
   `2014-Sep-Logic-3`
   a. It would activate different rows, but it would still work
   b. It would activate different rows, and would not work
   c. It would activate multiple rows, but would still work
   d. It would activate multiple rows, and would not work

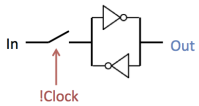
   An encoder could have multiple outputs active at the same time. E.g., if input #3 was selected then outputs 0 and 1 would both be true. This would cause the SRAM array to output data from rows 0 and 1 at the same time, and since they share output wires, their values would corrupt each other. The decoder was used because it has a one-hot output.

3. Which one of these circuits updates its output only when the clock goes from 1 to 0?

a. 

b. 

c. 

d. 

`2013-Jan-Logic-6`

a. !Clock / Clock
b. Clock / !Clock
c. Clock
d. !Clock

The signal reaches the first latch when clock is 1. Then when the clock drops to 0, the signal travels from the first latch to the second latch and sets the second latch. Thus the falling edge triggered flip-flop requires Clock, !Clock combination.

4. Will a MUX function correctly if more than one input is active (valid) at a time, and why?

`2016-Oct-Logic-2`

a. No, output is undefined or invalid
b. No, a MUX can only have one input at a time
c. Maybe, but it will depend on how the MUX is built internally
d. Yes, output is always the selected input

All the inputs to a MUX are valid; the MUX just selects which one to pass on to the output.

5. What will be the value of R12 after executing the following two instructions?

```
and R12, R12, R12
or R12, R12, R12
```

`2014-Dec-Logic-1`

a. 0000 (Zero)
b. FFFF (All ones)
c. It depends on the value of R12 before the first instruction
d. The value of R12 will not change.

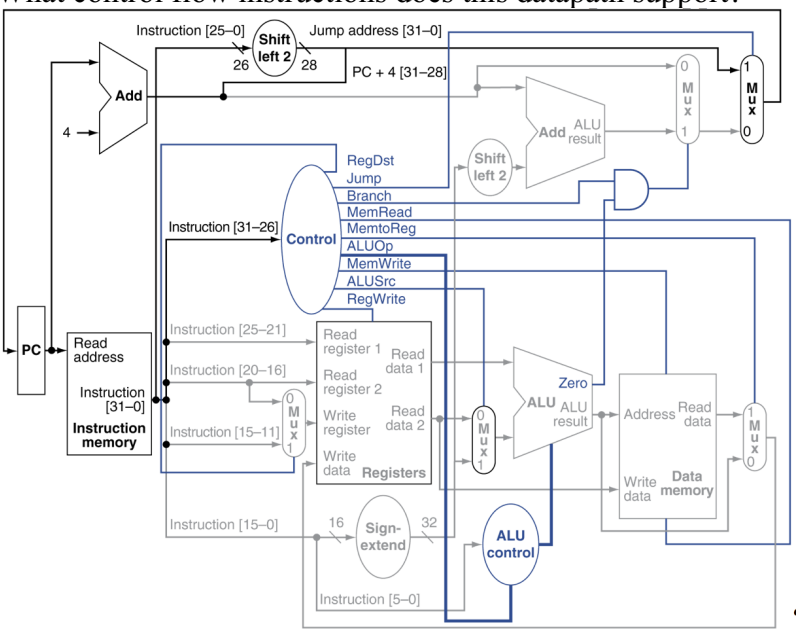
The logic operations AND and OR are idempotent, this means that A AND A = A, A OR A = A, therefore, the value will not change.

6. Why are binary circuits more immune to noise than analog circuit?
   2015-Oct-Logic-4
   a. Binary values are further apart in terms of voltage than analog values, so they are harder to confuse
   b. Binary circuits generate un-polluted outputs (e.g., 1 or 0 out) after each circuit, so the noise doesn't propagate through
   c. Binary circuits interpret values close to a 1 or 0 as a 1 or 0 even if they are not exactly 1 or 0
   d. All of the above

   All of the above.

5. Processor Control and Datapath

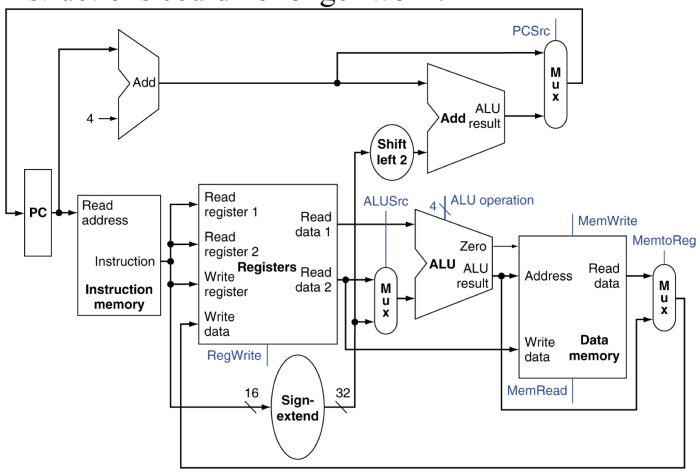
1. What control flow instructions does this datapath support?



   2017-Jan-Processor Control and Datapath-5
   a. BEQ, BNE
   b. J, BNE
   c. J, BEQ
   d. J, BEQ, BNE

   The AND gate will only allow us to take the PC+Immediate branch if ALU Zero is true, which means the two values are the same. That is, it only supports BEQ. We support Jump through the top path.

2. If the ALUSrc mux was removed, and Register Read data 2 was connected directly to the ALU, what instructions could no longer work?

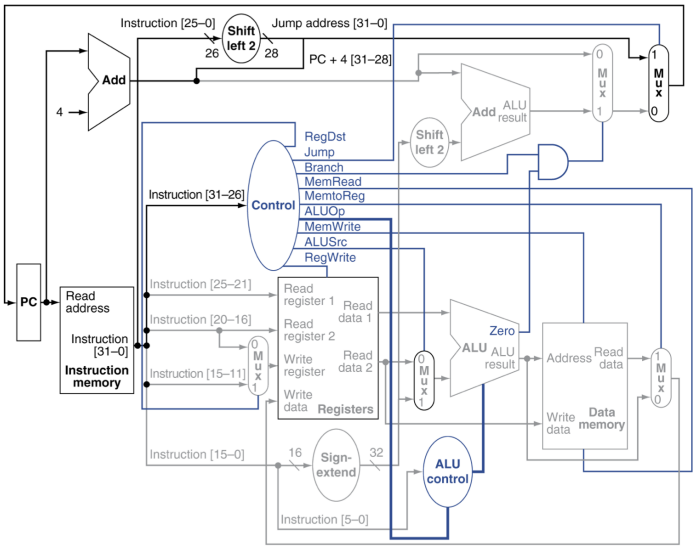


2013-Apr-Processor Control and Datapath-1

a. R-type
b. I-type
c. J-type
d. All of the above

I-type instructions require the data from the sign-extended immediate field. Changing this MUX would prevent that data from getting to the ALU.

3. Which part of the processor is storing state?



2015-Apr-Processor Control and Datapath-1

a. PC, registers, ALU
b. PC, registers, ALU, instruction memory
c. PC, registers, ALU, instruction memory, sign extension
d. PC, registers, instruction memory

The ALU is just combinational logic. It takes whatever its inputs are and generates outputs. The other components all store values and update them on the clock signal.

4. A single-cycle processor has three types of instructions: A takes 1ns, B takes 2ns, and C takes 10ns. A program with an instruction mix of 25% A, 70% B, and 5% C will run at what clock speed?
   `2017-Jan-Processor Control and Datapath-2`
   a. 1000 MHz (1 / 1ns)
   b. 465 MHz (1/2.15ns)
   c. 333 MHz (1/3ns)
   d. 100 MHz (1/10ns)

   The slowest instruction path dictates the clock speed.

5. What is the logic that determines whether PCSrc should select PC+4 (0) or the ALU result (1)?
   `2014-Sep-Processor Control and Datapath-4`
   a. (Zero AND bne) OR (Zero AND beq)
   b. (!Zero AND bne) OR (Zero AND beq)
   c. (Zero AND bne) OR (!Zero AND beq)
   d. (!Zero AND bne) OR (!Zero AND beq)

   The ALU result should be used if we want to branch. If the instruction is beq this means Zero is true and if it is bne this means Zero is false.

6. A processor has a broken sign-extender that always outputs 0s in the upper 16 bits. How would this limit what it can do?
   `2016-Oct-Processor Control and Datapath-2`
   a. lw and sw would always use the same address
   b. addi would not change the value
   c. You could not load 32-bit constants
   d. Conditional branches could only jump forwards

   lw and sw would not add an offset, so they would use whatever address was in the register file. Addi would add only positive values, but it would still change the value. You could still load 32-bit constants by using lui and addi. However, branches would no longer be able to go backwards as 16-bit negative two's complement offsets would not be properly sign-extended. (The postivie ones would be.)

6. Pipelining

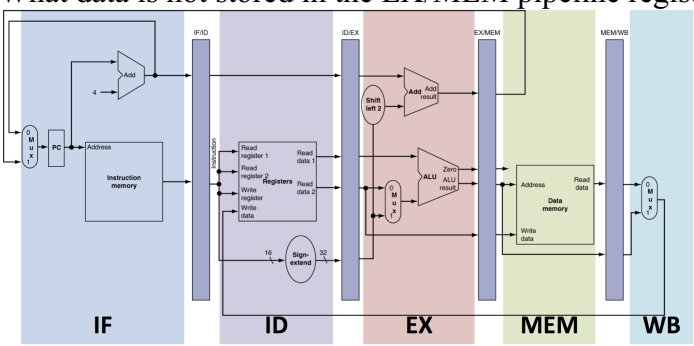
1. Does the pipelined processor keep the ISA's promise of in-order and atomic execution?
   `2015-Apr-Pipelining-6`
   a. Atomic: yes, In-order: yes
   b. Atomic: yes, In-order: no
   c. Atomic: no, In-order: yes
   d. Atomic: no, In-order: no

   The pipeline executes instructions in neither an atomic nor in-order manner. The instructions now happen in 5 separate pieces (not atomically) and don't happen in-order (e.g., one instruction's EX may happen before another instruction's MEM).

2. A single-cycle processor takes 100ns per instruction and can be perfectly divide up into pipeline stages and the pipeline can be kept completely full. However, each pipeline register has an overhead of 1ns. What degree of pipelining gives the best instruction throughput?
`2014-Sep-Pipelining-4`
    a. 1 stage (1 pipeline register at the end)
    b. 10 stages
    c. 100 stages
    d. 1000 stages

1000 stages will mean that you spend 100/1000=0.1ns computing per stage plus 1ns register overhead. This will give a cycle time of 1.1ns, which means you get an output every 1.1ns. 10 stages will mean you spend 100/10=10ns computing plus 1ns register overhead. This will give you a cycle time of 11ns, which means you get an output every 11ns. So the more stages the better for throughput as long as the pipeline is full.

3. What data is not stored in the EX/MEM pipeline register?



`2014-Apr-Pipelining-6`
    a. Register File 1 output
    b. Register File 2 output
    c. ALU result
    d. Branch address

We do not need to keep Register File 1 output in this stage since it has already been put into the ALU.

4. What is the speedup if you have an ideally pipelined processor with 15 stages, but due to control hazards you can only keep it full of instructions 66% of the time?
`2016-Aug-Pipelining-1`
    a. 0.66x
    b. 10x
    c. 15x
    d. 22.7x

An ideally pipelined 15-stage processor would be 15x faster, but if we can only keep it 66% full, then we will lose 33% of the performance, so we will be 10x faster .

5. Which of the following statements is not true about pipelined processors?
   `2014-Dec-Pipelining-1`
   a. Commercial processors today typically have 15-25 pipeline stages.
   b. It is possible to design processors with multiple parallel pipelines of different lengths.
   c. Doubling the pipeline length doubles the processing speed.
   d. Processors use separate instruction and data caches so they can load an instruction at the same time as they load data.

   If you make the pipeline longer it becomes increasingly harder to keep the pipeline full, so twice as long a pipeline will not run twice as fast.

6. What happens in the MEM stage for an addi instruction?
   `2013-Aug-Pipelining-5`
   a. Nothing
   b. Load the value
   c. Calculate the ALU result
   d. Write back the ALU result

   Nothing. The addi instruction does not access the data memory, so it does nothing in the MEM stage.

7. Hazards

1. Which of these does not address a structural hazard?
   `2015-Apr-Hazards-6`
   a. Having independent data and instruction memories
   b. Reading and writing the register file in the same cycle
   c. Extra comparator for branch decisions
   d. Forwarding from a later stage

   Data forwarding addresses a data hazard (the data is somewhere else). All the others add hardware or modify hardware to allow multiple instructions to execute at the same time.

2. What forwarding paths do we need to execute this code with no NOPs on a processor with a double-pumped register file?
   ```
   add R1, R2, R3
   lw R2, 0(R1)
   sw R2, 0(R3)
   sub R2, R1, R3
   ```
   `2014-Dec-Hazards-6`
   a. MEM-EX, WB-EX
   b. MEM-EX, WB-EX, WB-MEM
   c. MEM-EX, WB-MEM
   d. WB-EX, WB-MEM

   From add to lw we need MEM to EX, from lw to sw we need WB to MEM, and from add to sub we don't need anything.

3. Hazards are a result of what?
   2014-Apr-Hazards-2
   a. Needing to keep the semantics of atomic and sequential execution
   b. Executing instructions in parallel
   c. Executing instructions in pieces
   d. All of the above

   If we didn't have to keep the atomic and sequential semantics of the ISA then we wouldn't have any hazards. (The programmer would just have to figure it out.) But since we do need to keep those semantics, the parallel execution of pieces of instructions causes hazards.

4. An 8-stage pipeline processor requires NOPs to avoid hazards. The compiler is aware of this and inserts appropriate NOP instructions into the code. What would happen if the processed code now is run on a single-cycle processor?
   2015-Oct-Hazards-3
   a. The program would execute correctly
   b. Conditional branches would not work correctly, due to the zero output from the ALU
   c. The code would not work correctly because the NOPs are in the wrong place
   d. The code would not work correctly because there are too many NOPs

   Everything would execute correctly since a single-cycle processor does not have any hazards, however the NOPs would make the program run slower than necessary.

5. How many register values need to be forwarded in the following code on our standard 5-stage pipeline?
```
add R3, R4, R5
sub R2, R3, R4
beq R2, R3, done
```
   2013-Apr-Hazards-3
   a. 1
   b. 2
   c. 3
   d. 4

   R3 (add->sub), R2 (sub->beq), and R3 (add->beq)

6. If a program has 20% branches with 1 branch delay slot per branch, and the compiler can find an instruction to put in the branch delay slot every other time, what is the performance slowdown due to the branch delay slot?
   2014-Sep-Hazards-5
   a. 5%
   b. 10%
   c. 20%
   d. 50%

   Half of the 20% of the time it takes an extra cycle: 10%*2 + 90%*1 = 1.1 cycles per instruction.

8. Branch Prediction and Exceptions and Interrupts

1. If you have a branch predictor that keeps track of branch history based on the least significant bits of the PC and can store information for 16 separate branches (direct mapped, 16 entry branch predictor), what happens when you have a program with the following behavior:
   branch at 0010 1011 1010 1100 always not taken
   branch at 0010 1011 1110 1100 always taken

   2014-Apr-Branch Prediction and Exceptions and Interrupts-6
   a. The program crashes
   b. The branch predictor will only predict for one of the branches
   c. The branch predictor will confuse branches and predict incorrectly
   d. All of the above

   The predictor will use the same bits 1100 from both of the branches to keep track of branch behavior and as a result it won't be able to tell them apart. This will lead to a bad prediction, but it won't cause the computer to crash.

2. How many branches are correctly predicted with a 2-bit predictor initialized to strongly not taken?

```
for (i=0; i<100,000; i++) {
        if (i%2==1) { // branch 1: if i is odd
                // do something
        }
} // branch 2: loop
```

   2013-Dec-Branch Prediction and Exceptions and Interrupts-4
   a. branch 1: 50,000/100,000 branch 2: 99,997/100,000
   b. branch 1: 75,000/100,000 branch 2: 50,000/100,000
   c. branch 1: 50,000/100,000 branch 2: 99,998/100,000
   d. branch 1: 99,998/100,000 branch 2: 99,997/100,000

   Branch 1 will alternate TNTNTN because it happens whenever i is odd. The 2-bit predictor will therefore go between 00/01/00/01 but always predict not taken, which will mean it is 50% wrong. The outer loop will be wrong the first two times and the last time.

3. Which predictor will not have 50% accuracy for the if statement in this loop?

```
loop {
  i++ // increment variable i
  if (is_odd(i)) // is_odd() returns true if 'i' is odd
    b=b*2 // multiply b by 2;
}
```

   2016-Aug-Branch Prediction and Exceptions and Interrupts-5
   a. 1-bit predictor
   b. 2-bit predictor
   c. BTFNT
   d. Static

   The one-bit predictor will have 0% accuracy as it will always predict the last value which is wrong. The others will always predict one value which will be right half the time.

4. A program contains 1 million instructions, of which 15% are branches. If we run it on a CPU with a 20 cycles branch penalty and a branch predictor that is 85% accurate, how many cycles will be wasted due to mispredictions?
   `2015-Oct-Branch Prediction and Exceptions and Interrupts-3`
   a. 100 000 cycles
   b. 200 000 cycles
   c. 450 000 cycles
   d. 1 000 000 cycles

   This program contains 150 000 branches. 15% of them will be mispredicted, and each misprediction costs 20 cycles. 150 000 * 0.15 * 20 = 450 000 cycles wasted.

5. What do you have to do to instructions in the pipeline that are executed due to a missprediction?
   `2014-Sep-Branch Prediction and Exceptions and Interrupts-3`
   a. Nothing if they are memory instructions
   b. Prevent them from changing anything
   c. Insert instructions to reverse their effects
   d. All of the above

   They need to be prevented from changing anything. That is, they must not write to the register file, memory, or PC. If their writes are disabled then they effectively don't do anything.

6. The branch predictor has a table of multiple branch information. What is the wrong explaination of the information of each branch entry in the predictor?
   `2022-Jun-Branch Prediction and Exceptions and Interrupts-1`
   a. **PC:** Acts like a 'tag' in a cache. Used to compare with the current PC.
   b. **History:** Holds the information of past branch behavior for this PC
   c. **Valid:** Holds the decision on whether to branch or not for this PC
   d. **Target:** Holds the address to branch to, if the predictor decides to branch

   Valid bit holds whether this branch entry holds a valid value or not.

9. Input/Output

1. Why would you use DMA?
   `2022-Jun-Input/Output-1`
   a. Using DMA, the I/O operation can be overlapped with other computation
   b. DMA allows using normal load store instructions to control I/O devices
   c. DMA allows the I/O device to work faster
   d. DMA let's the processor know when the I/O operation is complete

   When using DMA, once the transfer is initiated, the application can perform other tasks.
   Using normal load and store is enabled by memory mapped I/O.
   The aim of DMA is not to make I/O operations faster.
   The mechanism to notify the processor is interrupts, not DMA.

2. Why would you choose to use interrupts instead of polling?
`2015-Apr-Input/Output-1`
    a. Polling has lower latency than interrupts
    b. Polling uses up less processor time
    c. Interrupts use up less processor time
    d. Interrupts have lower latency

Interrupts may take longer (since you have to go through the interrupt handler) but they allow the processor to do other things while it is waiting for I/O. Polling, however, will much more quickly let you know when data is available since you don't have to go through the interrupt handler, but you can't do anything else while you are polling.

3. Improvement of storage technology has changed I/O. What is incorrect of the following statements?
`2022-Jun-Input/Output-2`
    a. SSDs have no moving parts resulting in zero 'seek time' unlike spinning drives (HDD)
    b. The lack of seek time in SSDs enables better random I/O (read/write different parts of the disk) compared to HDDs
    c. Because SSDs are generally quicker (in latency) than HDDs, the amount of computation the CPU can do while the disk is working is less for the SSD.
    d. With very fast SSDs (comparable to memory speed), it is beneficial to change from the usual polling to interrupts, as the quicker SSDs will result in shorter interrupt handling times.

Usually interrupts are used because disks are slower compared to processors, and thus interrupts let the processor do other work while waiting for I/O. With quicker SSDs, the gap between storage and processor grows smaller, allowing quick-SSDs to be serviced using polling, without wasting too much processor cycles.

4. Which of the following is NOT a benefit of memory-mapped IO?
`2014-Dec-Input/Output-2`
    a. Faster
    b. Uses regular instructions to access IO
    c. Unifies all the device's memories
    d. Enables the use of virtual memory to protect IO devices

With memory-mapped IO you just need to know the address of the device and then do a regular load/store to access it. Memory-mapping is just a way to get the data, it does not change the performance per se. All the internal memories from the devices are unified since they are mapped to main memory address space.

5. What are the benefits of hard disks over flash memory?
`2014-Sep-Input/Output-5`
    a. Faster
    b. Cheaper
    c. Smaller
    d. None

Hard disks are still cheaper than flash, but they are much slower and larger.

6. What is hard about building communications busses?
   `2014-Sep-Input/Output-2`
   a. Using enough wires to get the bandwidth
   b. Synchronizing the signals on all the wires
   c. Finding space in servers
   d. All of the above

   Fast busses are hard to build because you have to keep the signals synchronized between all the wires.

10. Caches

1. Which property do caches not rely on to work well?
   `2022-Jan-Caches-2`
   a. Recently used data will be reused soon
   b. There are more reads than writes
   c. Nearby data will be reused soon
   d. The amount of data used at any one time is small

   Caches are designed to be effective for both loads and stores

2. How many lines will each way have in a 256kB, 4-way set associative cache with a 32-byte line size?
   `2014-Sep-Caches-3`
   a. 8192
   b. 2048
   c. 1024
   d. 512

   256kB / 32 bytes = 8192 lines total. The cache is 4-way associative, which means that each way will have 8192 / 4 = 2048 lines

3. What is a downside of making the cache line twice as large?
   `2015-Aug-Caches-4`
   a. Programs that access data sequentially will waste space
   b. Programs that access data randomly will waste space
   c. Tag comparisons are slower
   d. The percentage of bits spent on tag storage is higher

   Since the cache line is twice as large, every time you access a random address it will bring in twice as much data to the cache (the line). This means that, unless other random addresses are in the same line, you will only be able to have half as many random addresses in the cache since there are half as many lines.

4. A program is reading every word (4 bytes) consecutively of a very large (4GB) array. Which of the following cache configurations can achieve a hit ratio of more than 50%? Assume LRU replacement policy?
   `2017-Jan-Caches-3`
   a. Cache size: 16 B. Block size 4 B. Direct mapped
   b. Cache size: 8 MB. Block size 8 B. 2-way associative
   c. Cache size: 2 KB. Block size 32 B. Fully associative
   d. All of the above

   None of these caches are large, so they will not be able to fit all the data. Since the data is being read sequentially, if we have a large enough cache block size, then we can reuse the same block repeatedly. A 32B block will be reused 8 times for sequential words, giving a hit ratio of 7/8=87.5%.

5. Intel's Skylake processors have a 256KB, 16-way set-associative L2 cache with a 64 bytes cache line size. Assuming 32-bits addresses, how are the bits in the address used to access the cache?
   `2016-Oct-Caches-4`
   a. 26 bits for the tag, 6 bits to choose a byte within a cache line
   b. 6 bits for the tag, 18 bits for the index, 8 bits to choose a byte within a cache line
   c. 8 bits for the tag, 18 bits for the index, 6 bits to choose a byte within a cache line
   d. 18 bits for the tag, 8 bits for the index, 6 bits to choose a byte within a cache line

   There are 64 bytes per cache line. Therefore we need 6 bits in order to choose a byte within a cache line (because 2^6 = 64). Moreover, the cache is divided in 256KB / 64B / 16 = 256 sets, so we need 8 bits to index the cache. We use the remaining 18 bits for the tag.

6. What is the incorrect description about cache organization?
   `2022-Jun-Caches-1`
   a. Direct mapped caches result in more conflict misses, meaning that data at two different addresses may map to the same entry and push each other out.
   b. N-way Set associative caches can help solve the conflict miss of direct-mapped caches by putting N blocks in the same set.
   c. Fully-associative caches are the cheapest to access, as they only need one tag comparison per cache lookup
   d. Direct mapped caches do not need replacement policies, but set associative and fully associative caches do

   Fully-associative caches need to compare all blocks of the cacheline, and thus it is most expensive to access.

11. Virtual Memory

1. Which TLB/cache arrangement allows you to know if you have a hit in the cache without checking the TLB?
   `2013-Apr-Virtual Memory-3`
   a. Virtually indexed, virtually tagged
   b. Virtually indexed, physically tagged
   c. Physically indexed, virtually tagged
   d. Physically indexed, physically tagged

   Virtually indexed means you don't need the TLB translation to start looking in the cache, and the virtual tag means you can see if the hit is what you are looking for without needing the physical translation.

2. Which of the following statements is **not** true about virtual memory?
   `2022-Jun-Virtual Memory-1`
   a. VM provides security by allowing other programs to read each other's data.
   b. VM allows programs to use more memory than is physically available in the system.
   c. VM saves memory space by allowing programs to share common data instead of each program having their private copies.
   d. VM provides flexibility by using indirection to use non-contiguous physical memory in a contiguous virtual memory space.

   Applications being able to read each other's data is a major security issue. VM tries to prevent such issue by preventing programs from reading each other's data

3. Why would we prefer a single-level page table rather than a multi-level one?
   `2017-Jan-Virtual Memory-3`
   a. To reduce the total size of the page table
   b. To make translations faster
   c. To reduce the in-memory size of the page table
   d. None of the above

   Single-level page tables are faster for lookups, so we could increase the speed of translation.

4. What is/are the biggest reasons that are page faults so excruciatingly slow?
   `2015-Aug-Virtual Memory-5`
   a. The TLB has to be reloaded with the right page table data
   b. The data has to be accessed from the hard disk
   c. The OS has to choose which data to evict from DRAM
   d. All of the above

   While all of the above are true, the vast majority of the performance loss comes from accessing the data on the hard disk. The others are very fast in comparison.

5. What is *not* a benefit of virtual memory?
   `2014-Sep-Virtual Memory-1`
   a. Memory is protected from other applications
   b. Memory accesses are faster
   c. Memory appears larger than the installed DRAM
   d. Memory does not have to be used in contiguous chunks

   Memory accesses with virtual memory first have to be translated, which can be very slow if the address is not in the TLB.

6. What does the TLB do?
   `2017-Jan-Virtual Memory-2`
   a. It caches recently-used data
   b. It caches recently-used address translations
   c. It caches recently-used pages
   d. It caches recently-used instructions

   The TLB caches the recently-used virtual-to-physical address translations..

12. Parallelism

1. What does a lock do?
   `2014-Apr-Parallelism-1`
   a. Protect data so only one processor can access it at a time
   b. Synchronize data so that different caches see the same value
   c. Provide a way for a program to check if another processor is using the data
   d. All of the above

   All locks do is provide a way for a program to check if another processor doing something. They don't actually fix any synchronization, but they allow you to write a program that is synchronized.

2. The fastest computer in Sweden, Beskow at KTH, has 53,632 processor cores. What would be the speedup of a program that was 99.99% parallel?
   `2016-Aug-Parallelism-4`
   a. 5x
   b. 536x
   c. 8,400x
   d. 53,632x

   From Ahmdal's law s = 1/(1-0.9999+0.999/53632)=8,429.

3. My laptop has 2 CPU processor cores that run at 2GHz (billion cycles per second) and can each do 4 instructions per cycle and a GPU (graphics processor) with 16 processor cores that run at 1GHz and each do 2 instructions per cycle. You can use either the CPU or the GPU at any given time. What is the fastest rate I can run a program that is 50% parallelizable in GIPS (giga (billion) instructions per second)?
   `2014-Dec-Parallelism-6`
   a. 12 GIPS
   b. 17 GIPS
   c. 20 GIPS
   d. 32 GIPS

   Running the serial part on one CPU core gives us 50% at 4*2GIPS and the parallel part on the GPU gives us 50% at 16*2*1GIPS = 20 GIPS total. All on the GPU would give us 50% at 2*1GIPS and 50% at 16*2*1GIPS = 17 GIPS. All on the CPU would give us 50% at 4*2GIPS and 50% at 2*4*2GIPS = 12 GIPS.

4. The cache for processor 2 wants to load an address. What does the cache for processor 1 need to check to see if it needs to write back data and tell processor 2's cache to wait for it?
`2013-Jan-Parallelism-6`
    a. If it has the line (tag and valid) and it is dirty
    b. If it has the line (tag and valid) and it is not dirty
    c. If it has the line (tag and valid)
    d. If it has the dirty line (tag and dirty)

    Processor 1 needs to take action if it has a dirty(valid) version of the cache line. This means that if the cacheline exists in processor 1's cache (tag matches, and is valid) AND dirty bit is set, processor 1 needs to write it back to the shared cache (or the memory).

5. Which of the following designs to improve performance does not require multiple program counters (PC)?
`2015-Oct-Parallelism-2`
    a. Simultaneous multithreading (issuing instructions from multiple programs into the pipeline at the same time)
    b. Multicore processors (having multiple processors on one chip)
    c. Dual-issue processors (issuing multiple instructions from the same program into the pipeline at the same time)
    d. All of them require multiple PCs

    Dual-issue processors issue two instructions each cycle, so the PC simply counts up by 8. The others require multiple PCs to keep track of which instruction in each program is next.

6. A processor can do two ALU operations (add/sub/etc.) at the same time for two instructions that come one after the other. A program has 40% load/stores, and 60% ALU operations. 10% of the instructions depend on a subsequent instruction. What is the best IPC you would expect?
`2013-Apr-Parallelism-6`
    a. 1.5
    b. 1.6
    c. 1.8
    d. 1.9

    Both load/store and ALU operations require the ALU, so 90% of the time we can find two instructions to do on our two ALUs, or 2*0.9+1*0.1=1.9.

Make sure you circle the answers on the answer page.
Answers on other pages will NOT be graded.